Escola SENAI “Almirante Tamandaré”

Engenharia de Requisitos e Desenvolvimento de Software com IA

Utilize este documento para organizar e registrar os resultados finais do projeto. Preencha cada seção com informações produzidas pela equipe durante a atividade. Quando necessário, inclua tabelas, capturas de tela ou links para materiais complementares.

1. Identificação da equipe

Integrantes da equipe	Ivan Luís dos Santos Vinícius Kenji Higa Makimoto

2. Resumo do projeto

Descreva brevemente o objetivo do Gerenciador Financeiro, o público que pretende atender e os principais problemas que a aplicação busca resolver.

Resumo do projeto:

O Gerenciador Financeiro é uma aplicação web local para registrar e acompanhar receitas, despesas, retiradas e transferências internas, destinada a pessoas ou pequenos negócios que necessitam organizar suas movimentações. Resolve a dispersão de registros, a dificuldade de identificar saldos realizados e previstos e o acompanhamento de vencimentos, categorias e relatórios. Não realiza operações bancárias reais.

3. Levantamento de requisitos

Resuma como a equipe investigou as necessidades do cliente. Registre as principais respostas, dúvidas identificadas e decisões tomadas.

Principais necessidades identificadas:

A documentação de requisitos e os prompts de interface descrevem necessidades de controle de movimentações, visualização de saldos e balanços, classificação por categorias, contas, filtros, relatórios e uso local. O pacote contém especificações das etapas 1 a 4, prompt de frontend e registros de ajustes. Não foram encontrados registros completos de entrevistas com um cliente; portanto, não é possível atribuir respostas a entrevistados específicos.

Dúvidas, ambiguidades e decisões confirmadas:

Decisões documentadas: execução local em PHP; persistência em arquivos JSON fora da pasta pública; receitas somam e despesas subtraem; apenas lançamentos pagos integram o saldo realizado. O README da versão entregue estabelece que transferências internas exigem origem e destino e não alteram o saldo consolidado. Há divergências entre propostas anteriores e o código final: recorrências sem término e despesas fixas mensais aparecem em solicitações, mas o README limita séries novas à quantidade definida e descreve regra fixa somente para receitas. A equipe deve validar essas decisões.

4. Especificação dos requisitos

4.1 Requisitos funcionais (RF)

Liste as funcionalidades que o sistema deve executar. Use identificadores como RF01, RF02 e descreva cada requisito de forma clara.

Código	Descrição do requisito	Prioridade
RF01	Autenticar usuários e cadastrar o primeiro administrador	Essencial
RF02	Cadastrar, editar, excluir e consultar receitas, despesas e transferências	Essencial
RF03	Gerenciar contas e categorias	Essencial
RF04	Exibir dashboard, saldos e previsão financeira	Essencial
RF05	Filtrar, ordenar e consultar transações	Essencial
RF06	Gerar relatórios mensais e anuais	Essencial
RF07	Gerenciar lançamentos parcelados e recorrentes com quantidade definida	Desejável
RF08	Gerenciar receitas fixas mensais e estados por competência	Desejável
RF09	Administrar usuários e permissões	Desejável

4.2 Requisitos não funcionais (RNF)

Registre condições de qualidade e restrições, como funcionamento local, facilidade de uso, responsividade e manutenção.

Requisitos não funcionais:

RNF01 — Execução local em PHP 8.1+ com servidor embutido ou Apache/XAMPP.
RNF02 — Interface em português, com valores em reais e layout responsivo.
RNF03 — Persistência local em JSON, com backups anteriores à gravação e arquivos fora de public/.
RNF04 — Segurança com sessões, hash de senha, token CSRF, validação no servidor e bloqueio/gravação atômica.
RNF05 — Separação de interface, API, serviços e repositórios para manutenção.
RNF06 — Sem dependência obrigatória de CDN; inadequado para muitos acessos simultâneos ou publicação pública sem revisão.

4.3 Regras de negócio (RN)

Descreva condições que determinam como o sistema deve funcionar, incluindo validações e restrições.

Código	Regra de negócio
RN01	Somente transações pagas afetam o saldo realizado; pendentes/vencidas entram na previsão.

RN02	Receitas aumentam e despesas reduzem o saldo; valores devem ser maiores que zero.
RN03	Transferências internas entre contas reduzem origem e aumentam destino, sem alterar o consolidado.
RN04	Movimentação pendente com vencimento anterior à data atual é exibida como vencida.
RN05	Receita fixa é exclusiva de receita e não pode coexistir com parcelamento/recorrência.
RN06	Séries novas de parcelas e recorrências têm quantidade definida; edição admite escopo da ocorrência/série.
RN07	Administrador gerencia usuários e categorias; permissões definitivas requerem validação.

4.4 Histórias de usuário e critérios de aceite

Para cada funcionalidade importante, registre a história de usuário e condições objetivas que permitam verificar se ela foi implementada corretamente.

História de usuário: Como [perfil], quero [ação], para [benefício].

HU01: Como usuário, quero registrar receitas e despesas para acompanhar minhas finanças.
HU02: Como usuário, quero consultar saldo realizado e previsto para planejar pagamentos.
HU03: Como usuário, quero filtrar transações por período, tipo e categoria para localizar lançamentos.
HU04: Como usuário, quero transferir valores entre minhas contas para manter saldos corretos.
HU05: Como administrador, quero gerenciar usuários e categorias para organizar o uso do sistema.

Critérios de aceite:

CA01: valor inválido ou não positivo é recusado.
CA02: lançamento pendente não altera saldo realizado; pago altera.
CA03: transferência interna altera saldos das contas envolvidas sem mudar o total.
CA04: lançamento vencido aparece com situação correspondente.
CA05: filtros e relatórios apresentam dados coerentes com período e situação.
CA06: dados persistem após atualização da página e permanecem fora da pasta pública. Estes são critérios verificáveis; não equivalem a testes executados.

5. Escopo e priorização

Informe quais funcionalidades fazem parte da versão entregue, quais ficaram para uma versão futura e quais foram consideradas fora do escopo.

Funcionalidades implementadas:

Evidências no código: páginas de login, dashboard, transações, contas, relatórios e configurações; endpoints de autenticação, usuários, categorias, contas, transações e dashboard; cálculo de saldos e relatórios; receitas fixas; séries finitas; operações em arquivos JSON e backups; controles de sessão, senha e CSRF descritos no

README. Os registros de ajustes também mencionam ordenação de tabelas e confirmação de exclusão, sujeitos a teste visual.

Funcionalidades não implementadas e justificativa:

Não comprovados como concluídos: execução e testes em ambiente PHP real; integração bancária, pagamentos e transferências reais (fora do escopo); publicação multiusuário em produção; recorrência nova sem término e despesas fixas mensais (solicitadas em materiais, mas não confirmadas como regra final no README); matriz definitiva de permissões. A ausência de testes documentados impede afirmar que todas as telas estão livres de falhas.

6. Arquitetura e tecnologias

Descreva as tecnologias utilizadas, a estrutura principal de pastas/arquivos e a forma de armazenamento local dos dados.

Tecnologias e justificativas:

Backend em PHP 8.1+ para páginas, APIs, autenticação e regras de negócio; HTML/CSS/JavaScript locais para interface e interações AJAX com fetch; arquivos JSON para persistência simples sem servidor de banco de dados. O código separa serviços financeiros e repositório de armazenamento. Bibliotecas por CDN do protótipo visual foram evitadas na aplicação local, segundo o README.

Estrutura do projeto e armazenamento dos dados:

public/: páginas PHP, endpoints public/api/ e assets/css, assets/js. app/: bootstrap, controllers, helpers, repositories/JsonRepository.php e services/FinancialService.php. config/app.php: caminhos e configurações. storage/data/: arquivos JSON; storage/backups/: cópias antes das alterações. A pasta storage/ deve permanecer fora de public/. O pacote também inclui especificações em PDF, prompt de frontend e registro de ajustes.

Como executar a aplicação localmente:

1. Instalar PHP 8.1 ou superior e verificar com php -v.
2. Abrir o terminal na pasta gerenciador-financeiro.
3. Executar: php -S 127.0.0.1:8000 -t public
4. Acessar http://127.0.0.1:8000; configurar o primeiro administrador e entrar com as credenciais criadas.
Alternativa XAMPP: copiar a pasta para C:\xampp\htdocs\gerenciador-financeiro, iniciar Apache e abrir http://localhost/gerenciador-financeiro/public/. Verificar permissões de escrita e .htaccess. Instruções documentadas, não executadas neste ambiente.

7. Uso de IA e engenharia de prompts

Registre os prompts mais relevantes utilizados durante o projeto, o resultado obtido e os ajustes feitos pela equipe. Inclua mais linhas ou anexe um registro complementar, se necessário.

Etapa	Prompt utilizado	Resultado obtido	Ajustes / avaliação
Requisitos	Especificar necessidades e regras de um gerenciador financeiro local.	Documentos de requisitos das etapas 1 a 4.	Revisar ambiguidades de transferências e recorrência.
Protótipo visual	Criar Ciclo 1 com login visual, shell responsivo e Dashboard, sem backend.	Prompt de frontend para Stitch e pacote de layout.	Distinguir protótipo de implementação PHP.
Ajustes de interface	Adicionar ordenação de tabelas, menu Nova movimentação e legenda dos gráficos.	Registro de solicitações e respostas de atualização.	Conferir visualmente o resultado no navegador.
Regras de lançamentos	Permitir despesa fixa e exclusão com opções de alcance da série.	Solicitações registradas em ajustes.txt.	Comparar pedidos com README/código final; confirmar o que foi implementado.

8. Reflexão final da equipe

Explique o que a equipe aprendeu sobre levantamento de requisitos, elaboração de prompts, desenvolvimento com IA e validação humana. Cite uma dificuldade e como ela foi resolvida.

Reflexão da equipe:

O projeto evidencia a importância de transformar pedidos de interface em regras testáveis e de revisar resultados gerados com apoio de IA. A comparação entre prompt inicial, ajustes e implementação revela uma dificuldade concreta: requisitos de transferências, recorrência e despesas fixas mudaram ao longo do trabalho. A forma recomendada de resolver é consolidar uma versão única dos requisitos e validar os fluxos no navegador. O pacote não contém relato assinado da equipe nem resultados completos de testes; esta reflexão é uma síntese técnica baseada nos artefatos, a ser revisada pelos integrantes.

11. Checklist da entrega

☐ Documento preenchido e revisado pela equipe.

☐ Código-fonte completo da aplicação incluído.

☐ Aplicação executa localmente e respeita as restrições do projeto.

☐ Registro dos prompts utilizados incluído.

☐ Relatório de testes preenchido.

☐ Capturas de tela ou outras evidências incluídas.

☐ Arquivos organizados e identificados com nome da equipe.

12. Instruções para envio do projeto

Antes de enviar: organize todos os materiais em uma única pasta. Inclua este documento preenchido, o código-fonte completo e os arquivos necessários para executar a aplicação. Não inclua pastas temporárias ou dependências que possam ser instaladas novamente, salvo orientação do professor.

Compactação: comprima a pasta do projeto em um arquivo .ZIP. Verifique se o ZIP contém os arquivos e pastas necessários e se o projeto pode ser executado seguindo as instruções registradas na seção 6.

Nome do arquivo: use o padrão Turma_Equipe_TaskFlow.zip e identifique também o documento preenchido, por exemplo: Turma_Equipe_TaskFlow_Entrega.docx.

Envio: envie o arquivo ZIP e este documento pelo ambiente de entrega indicado pelo(a) professor(a), até o prazo informado em aula. Se o ambiente aceitar apenas um arquivo, reúna o documento e o projeto em um único ZIP.

Conferência final: antes de finalizar o envio, confirme que o arquivo foi anexado corretamente e que não está vazio ou corrompido. Se houver um link para repositório, ele deve estar acessível e conter o projeto completo; o link não substitui os arquivos, a menos que o(a) professor(a) autorize.